

Task Manager with File Persistence

An x86 Assembly Language Project

Assembly Language (x86) | NASM Assembler | DOSBox Emulator

Submitted by :

[Muhammad Ahmad]
[Alishba Haroon]

Roll No : [SU92-BASIM-F23-135]
[SU92-BASIM-F23-116]

Course : Computer Organization And Assembly Language

Session : [Fall 2023]

Date : 10 May 2026

GitHub Link : <https://github.com/AHMAD-0702/Computer-Organization-And-Assembly-Language>

1. Introduction

This report documents the design and implementation of a Task Manager with File Persistence — a console-based application developed entirely in x86 Assembly Language using the NASM (Netwide Assembler) toolchain and executed within the DOSBox DOS emulator environment.

The primary purpose of this project is to demonstrate the practical application of low-level programming concepts, specifically DOS interrupt-driven input/output and file handling operations. Task management is a universally relevant domain; by implementing it at the assembly level, the project bridges the conceptual gap between high-level application logic and the hardware-level mechanics that underpin all modern computing.

1.1 Why File Handling in Low-Level Programming?

File handling is a cornerstone of any practical application. In high-level languages, file operations are abstracted behind rich library functions. In assembly language, the programmer must directly invoke operating system services — in this case DOS INT 21h interrupts — to perform every byte-level read, write, and seek operation. This exposes the full machinery of I/O: file descriptors (handles), buffer management, pointer arithmetic, and error checking using CPU flags.

Mastering file I/O at this level provides deep insight into how data persists beyond program termination, how the OS mediates access to disk storage, and why concepts such as buffered writes and explicit file closure are critical for data integrity.

2. System Requirements

The following software and hardware components are required to assemble, link, and execute this project:

Component	Details
Operating System	MS-DOS 6.x (emulated via DOSBox 0.74-3 or later)
Assembler	NASM (Netwide Assembler) v2.15+ — generates flat binary / .com / .exe
Emulator	DOSBox 0.74-3 — provides a complete x86 DOS environment on modern OS
Host OS	Windows 10/11, Linux, or macOS (to run DOSBox)
Memory	Minimum 640 KB conventional memory (DOS limit; DOSBox default)
Storage	At least 5 MB free disk space for DOSBox drive image and data.txt
Text Editor	VS Code, Notepad++, or any plain-text editor for .asm source files

2.1 Assembly and Execution Commands

```
; Inside DOSBox, mount and run:
mount c c:\assembly
c:
nasm task.asm -o task.com
task.com
```

3. Technical Implementation

The program is structured around three principal functional modules: the Menu and Input Handling subsystem, the File Write subsystem, and the File Read (Display) subsystem. These modules interact via shared data segments and a common file handle stored in a memory variable.

3.1 Program Entry and Initialization

Upon execution, the DOS loader places the program into memory and transfers control to the entry point. The code segment (CS) and data segment (DS) registers are configured, and the stack pointer (SP) is initialised. The application then falls through to the main menu loop.

3.2 Menu and Input Handling

The main menu is rendered using INT 21h / AH=09h, which outputs a '\$'-terminated ASCII string to the console. The menu presents three choices:

- 1 — Add Task
- 2 — View Tasks
- 3 — Exit

User input is captured via INT 21h / AH=0Ah (Buffered Keyboard Input), which reads a complete line including carriage return into a predefined buffer. The first character of the buffer is compared using the CMP instruction, and a series of conditional near jumps (JE, JNZ) routes execution to the appropriate handler. Near jumps are employed throughout to avoid the 'short jump out of range' displacement error that would arise with direct short-jump instructions across code-dense segments.

3.3 File Writing — Add Task

When the user selects option 1, the following sequence executes:

- The program prompts for a priority value (e.g., High / Medium / Low) using INT 21h / AH=09h followed by AH=0Ah for input.
- It then prompts for the task description string.
- The file data.txt is opened for writing using INT 21h / AH=3Dh with access mode 01h (write). If the file does not yet exist, it is first created via AH=3Ch.
- The file pointer is moved to the end of file using INT 21h / AH=42h (LSEEK) with AL=02h (seek from EOF) and CX:DX = 0000:0000, ensuring all new entries are appended rather than overwriting existing data.
- The formatted entry string [Priority][Task Description] followed by a carriage-return/line-feed (CR+LF) pair is written to the file using INT 21h / AH=40h, with CX set to the byte count of the record.
- The file handle is closed via INT 21h / AH=3Eh to flush OS buffers and release the handle.

3.4 File Reading — View Tasks

When option 2 is selected:

- The file data.txt is opened in read-only mode using INT 21h / AH=3Dh with access mode 00h.
- A single-byte read loop is entered: each iteration calls INT 21h / AH=3Fh with CX=1 to read one byte into a one-byte buffer.
- The return value in AX is checked; AX=0 signals end-of-file, terminating the loop.
- Each byte is echoed to the console using INT 21h / AH=02h (character output) or by constructing a single-character '\$'-terminated string for AH=09h output.
- After the loop, the file is closed with AH=3Eh.

3.5 Data Format

Each task record persisted in data.txt adheres to the following pipe-delimited format:

```
[Priority]|[Task Description]<CR><LF>
```

For example:

```
High|Submit the Computer Architecture assignment
Low|Buy groceries from the market
```

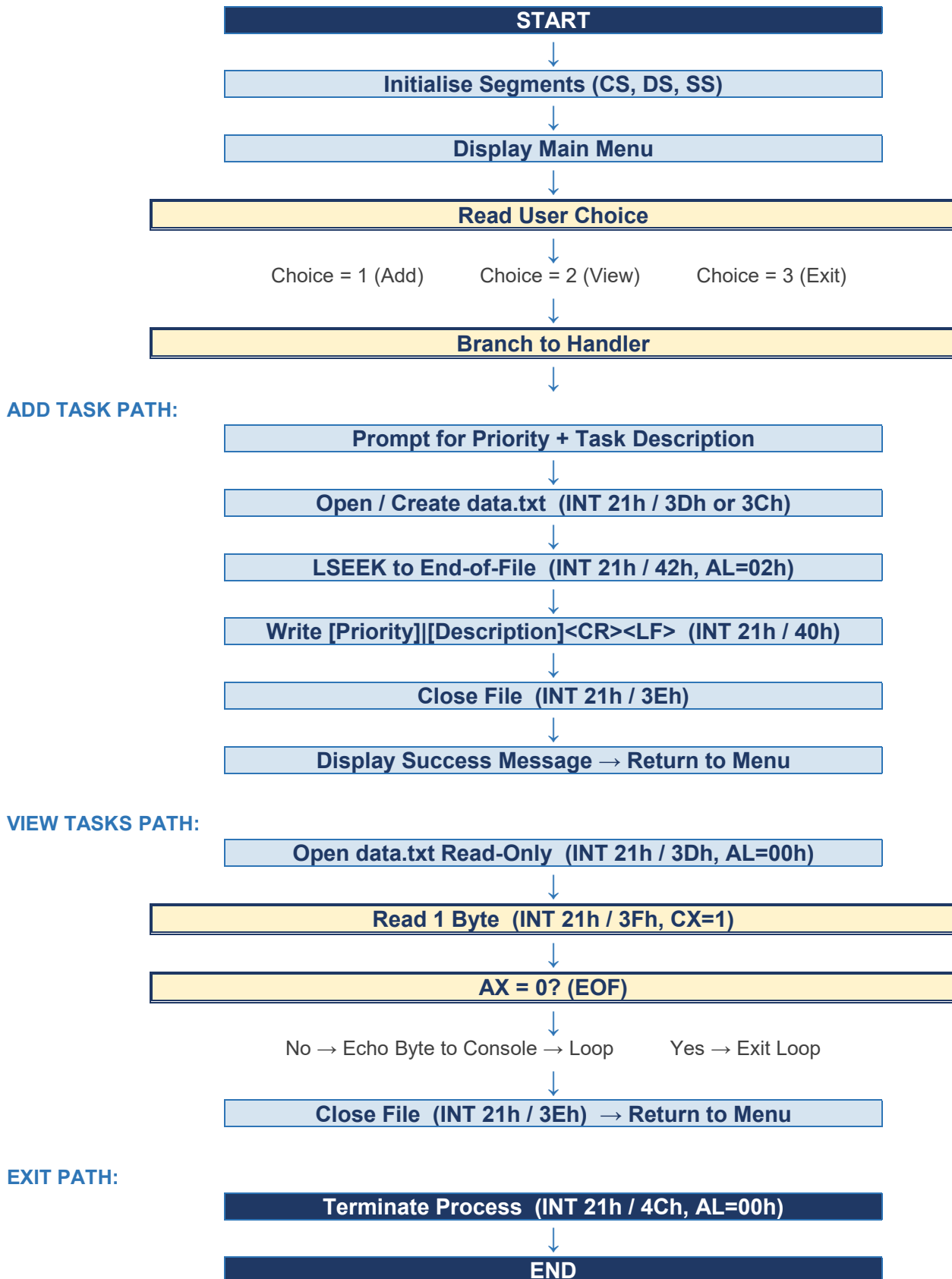
This delimiter-based format allows future enhancements (such as a parser that splits on '|') to easily extract individual fields without requiring a fixed-width record scheme.

3.6 Register Usage Summary

Register	Role in This Program
AX	Holds the interrupt function number (AH) and return values (file handle, bytes read).
BX	Stores the open file handle returned by INT 21h / 3Dh and 3Ch.
CX	Specifies byte count for read/write operations; upper word for LSEEK.
DX	Points to DS-relative buffer/string addresses for all I/O calls; lower word for LSEEK.
DS:DX	Effective address pair for string and buffer arguments.
SI / DI	Used as index registers for byte-by-byte buffer traversal.
Flags (CF)	Carry flag set on error by DOS interrupts; tested with JC for error branching.

4. Program Flowchart

The flowchart below illustrates the complete logical flow of the Task Manager, from program start through all user-interaction branches to final file closure and program termination.



Note: Screenshots of each stage — the menu, adding a task, and the output of type data.txt — should be inserted here as figures in the printed version of this report.

5. DOS Interrupts Reference Table

The following table catalogues every INT 21h service call employed in this project, together with the register arguments used and the purpose served within the application:

INT 21h / AH	Function Name	Purpose / Description
21h / 3Ch	Create File (CREAT)	Creates a new file or truncates an existing file; returns file handle in AX.
21h / 3Dh	Open File	Opens an existing file in read, write, or read-write mode; returns file handle in AX.
21h / 3Eh	Close File Handle	Flushes buffers and releases the file handle; essential to prevent data loss.
21h / 3Fh	Read File / Device	Reads a specified number of bytes from an open file into a memory buffer.
21h / 40h	Write File / Device	Writes a specified number of bytes from a memory buffer to an open file.
21h / 42h	Move File Pointer (LSEEK)	Repositions the read/write pointer; used here to seek to end-of-file for appending.
21h / 09h	Print String	Outputs a '\$'-terminated string to the console for menu and prompt display.
21h / 0Ah	Buffered Keyboard Input	Reads a line from the keyboard, including the character count, into a buffer.
21h / 4Ch	Terminate Program	Terminates the program and returns an exit code to DOS.

All interrupt calls conform to the DOS Application Binary Interface (ABI): function code is loaded into AH before INT 21h is issued; error status is signalled by the Carry Flag (CF=1) on return, with the error code in AX.

6. Conclusion

This project has provided comprehensive, hands-on experience with x86 assembly language programming at the systems level. The following key competencies and insights were developed and reinforced throughout the implementation:

6.1 CPU Register Management

Working directly with 16-bit general-purpose registers (AX, BX, CX, DX) and segment registers (CS, DS, SS, ES) has solidified the understanding that a CPU has only a very small number of high-speed storage locations. Efficient register allocation — loading a value once, using it across several instructions, and only then storing it

to memory — is essential for correct, performant code. The fact that DOS interrupt calls overwrite specific registers (especially AX on return) required careful save/restore discipline using PUSH and POP instructions.

6.2 Disk I/O via Software Interrupts

The project demonstrated that all disk access in a DOS environment is mediated by the operating system through software interrupts. The programmer never addresses hardware registers of the disk controller directly; instead, INT 21h acts as an abstraction layer, translating high-level requests (open, read, write, seek) into low-level DMA and IRQ operations managed by DOS and the BIOS. This layered model is the conceptual ancestor of modern system calls in POSIX and Win32.

6.3 File Persistence and Data Integrity

The use of LSEEK (INT 21h / 42h) to position the file pointer at the end of file before every write operation ensures that the append semantics are honoured even if the file had been previously opened and partially written. Equally important is the explicit close operation (INT 21h / 3Eh): omitting this call can leave data residing in the OS write-back cache rather than on disk, leading to corrupted or incomplete records.

6.4 Byte-Level Data Structures

Designing the pipe-delimited [Priority][Task Description]<CR><LF> record format from first principles illustrated that even the simplest structured data requires deliberate choices about delimiters, line endings, and termination conditions. These considerations scale directly to understanding formats such as CSV, fixed-width records, and binary file structures used in production software.

6.5 Near Jump Strategy

Replacing short (2-byte) conditional jumps with near (3-byte) jumps wherever the target address exceeded the +127/-128-byte displacement range of a short jump resolved the 'short jump out of range' assembler error. This practical encounter with jump displacement constraints deepened the understanding of how branch encoding is tied directly to instruction encoding size and memory layout.

In summary, the Task Manager with File Persistence project successfully demonstrates that even a relatively complex, user-facing application — featuring persistent storage, structured data, and an interactive menu — can be realised at the most fundamental level of the computing stack. The experience reinforces that all higher-level abstractions ultimately rest on operations precisely like those coded here: register loads, interrupt calls, and byte-by-byte I/O.